

PROYECTO:

EAMMRA

Dirección del proyecto: *Igor PRARIO*

Co-Dirección del proyecto: *Silvia BLANC*

**Integración de subsistemas para el prototipo de
EAMMRA.**

(En etapa de redacción)

Mariano CINQUINI, Patricio BOS, Igor PRARIO y Rui MARQUES ROJO.

División Acústica Submarina

Informe Técnico AS 02/19

Abril 2019



DIIV
DIRECCIÓN DE
INVESTIGACIÓN DE LA ARMADA



UNIDEF
UNIDAD EJECUTORA DE
INVESTIGACIÓN Y DESARROLLO
ESTRATÉGICOS PARA LA DEFENSA
CONICET/MINDEF

Integración de subsistemas para el prototipo de EAMMRA.

Mariano CINQUINI, Patricio BOS, Igor PRARIO y Rui MARQUES ROJO

RESUMEN

Completar...

ABSTRACT

Complete...

Este trabajo es parte del Proyecto: ‘Dispersión acústica’,
que se lleva a cabo en la División Acústica Submarina
de la Dirección de Investigación de la Armada (DIIV) y
la Unidad Ejecutora de Investigación y Desarrollo Estratégicos
para la Defensa (UNIDEF), dependiente de CONICET/MinDef.

AGRADECIMIENTOS

Se hace explícito el agradecimiento a:



Completar...

ÍNDICE

I.	INTRODUCCIÓN	8
II.	DESCRIPCIÓN DE LOS SUBSISTEMAS QUE COMPONEN EAMMRA	10
II.1	Sensores	10
II.1.a	Hidrófonos y preamplificador	10
II.1.b	Sensores de Temperatura	11
II.1.c	Anemómetro y GPS	12
II.2	Instrumental de adquisición	12
II.2.a	Placa de adquisición	12
II.2.b	Instrumental integrado en los sensores	14
II.3	Unidad de procesamiento	14
II.4	Unidad de control	15
II.5	Alimentación	16
III.	DESCRIPCIÓN DE LAS ACTIVIDADES DEL PROTOTIPO	17
III.1	Ciclo de funcionamiento sintético	17
III.1.a	Ciclo completo	17
III.1.b	Ciclo simplificado	17
III.1.c	Presencia de usuario conectado por SSH	17
III.1.d	Casos de falla	17
III.2	Diagrama de flujo completo	17
IV.	MÓDULOS PRINCIPALES DE SOFTWARE	18
IV.1	Unidad de procesamiento	18
IV.1.a	Sistema operativo y lenguaje de programación utilizados	18
IV.1.b	<i>Scripts</i> de Python	18
IV.1.c	Interfaz de usuario	19
IV.2	Unidad de control	20
IV.2.a	Módulo RTC	20
IV.2.b	Módulo CUBIE	22
IV.3	Protocolos específicos	24
IV.3.a	Temperatura - <i>1 wire</i>	24
IV.3.b	GPS. Mensajes NMEA sobre protocolo serie	25
IV.3.c	Anemómetro	25
V.	CONEXIONADO FÍSICO	26
V.1	Diagrama funcional en bloques	26
V.2	Puertos y conectores	26
V.2.a	Sensores	26
V.2.b	Adquisición	26
V.2.c	Unidad de procesamiento	26
V.2.d	Unidad de control	26
V.2.e	Alimentación	26
V.3	<i>Housing</i> y diseño mecánico	26
VI.	PRUEBAS DE LABORATORIO	27
VI.1	Pruebas de uso y funcionamiento	27
VI.2	Estudio de consumo y duración de la batería	27
VI.3	Estudio de llenado de la memoria SD	27
VI.4	Casos de falla	27

VI.5	Diseño de solución óptima en función de ciclos de batería y disco	27
VII.	CONCLUSIONES	28
	REFERENCIAS	29
	APÉNDICES	30
I.	<i>Scripts</i> computacionales.	30
II.	Circuitos de electrónica adicional (borneras).	30
III.	Especificaciones de partes constitutivas.	30
IV.	Planos.	30

GLOSARIO DE SIGLAS

ARA	Armada Argentina
COFS	Comando de la Fuerza de Submarinos
CENPAT	Centro Nacional Patagónico
CONICET	Consejo Nacional de Investigaciones Científicas y Técnicas
DIIV	Dirección de Investigación de la Armada
GSI	Golfo San Jorge
HID	Estación hidrofónica
I+D	Investigación y Desarrollo
INIDEP	Instituto Nacional de Investigación y Desarrollo Pesquero
MinCyT	Ministerio de Ciencia, Tecnología e Innovación Productiva
MinDef	Ministerio de Defensa
N	Abundancia Numérica (del inglés, Numerical Abundance)
NL	Nivel de Ruido
NL_a	Ruido Ambiente marino propiamente dicho
NL_p	Ruido Propio asociado al sistema de medición
PIDDEF	Proyecto de Investigación y Desarrollo para la Defensa
RFI	Estación de muestreo fitoplanctónico para monitoreo con ultrasonido
S_V	Fuerza de Dispersión acústica (del inglés, <i>acoustic Scattering Strength</i>)
SHN	Servicio de Hidrografía Naval
SIID	Servicio Naval de Investigación y Desarrollo
SONAR	Del inglés, acrónimo de (<i>SOund Navigation And Range</i>)
TL	Pérdidas por Transmisión acústica (<i>Transmission Loss</i>)
TS	Fuerza de Blanco (<i>Target Strength</i>)
UNIDEF	Unidad Ejecutora de Investigación y Desarrollo Estratégicos para la Defensa

I. INTRODUCCIÓN

El prototipo de la Estación Autónoma Marítima para el Monitoreo de Ruido Ambiente EAMMRA es, en esencia, un sistema electrónico modular alojado en una boya de superficie. Consta de sensores, instrumental de adquisición y unidades de procesamiento y control, genéricamente llamados *subsistemas*. En esta etapa del desarrollo, las funciones principales del prototipo incluyen:

- el registro de señales hidroacústicas a dos profundidades diferentes,
- el registro de la posición GPS y de variables ambientales *in-situ*, tales como temperatura en superficie, temperatura en puntos de la columna de agua y velocidad del viento,
- almacenamiento y procesamiento de los datos acústicos,
- interacción con un usuario para operar las funciones de la estación, para la recuperación de los datos almacenados y para la prueba y depuración del sistema en su conjunto.

En la figura 1 se pueden apreciar 5 módulos o subsistemas que actualmente componen la estación de monitoreo EAMMRA, esquemáticamente agrupados en bloques. El subsistema de alimentación provee energía eléctrica a los demás módulos, lo que se representa a través de flechas color verde. En color negro se pueden observar los bloques de los subsistemas de control y procesamiento que cuentan con computadoras programables y *software* desarrollado *ad-hoc*. Mediante flechas color negro se indica el sentido en que los módulos envían órdenes o comandos de control a otros módulos para la toma de datos. Los bloques de los subsistemas de sensores y de adquisición se pueden observar en color azul y son los responsables de generar la información y acondicionarla para su almacenamiento, respectivamente. También en color azul se representa el sentido en que la información de interés para el monitoreo es transmitida dentro del sistema.

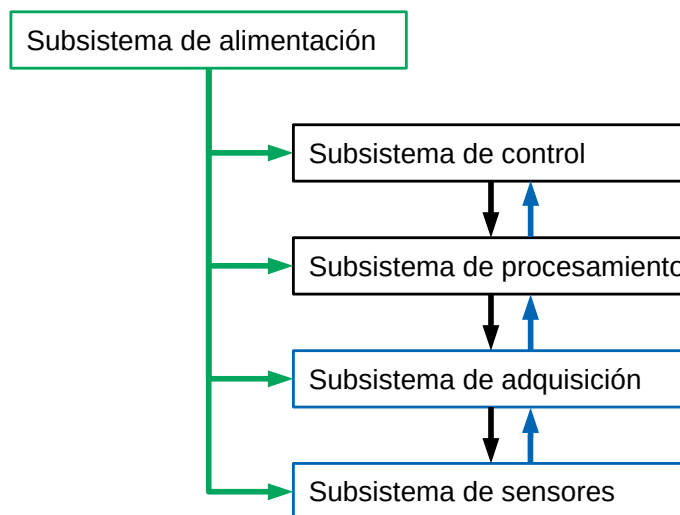


FIG. 1. Diagrama en bloques de los subsistemas que componen EAMMRA. Se indican en color verde las conexiones de alimentación eléctrica, en color negro las conexiones de comando y control y en color azul el sentido en que la información de interés es transmitida.

El subsistema de control es el encargado de gestionar la ejecución de cada una de las tareas que efectúa el prototipo y establecer un ciclo de trabajo. Este módulo debe constar de un hardware robusto y estar preparado para manipular eventuales casos de falla. Algunas de las tareas que tendrá a su cargo

son el comando de la adquisición y procesamiento de información, la integración coherente de los datos provenientes de los distintos sensores y su almacenamiento en una unidad física de memoria no volátil.

Las distintas características del instrumental de adquisición exigen que, en casos como el del registro de señales acústicas, se requiera de un hardware adicional, capaz de integrarse con los demás subsistemas y de ejecutar un sistema operativo de propósitos generales basado en una distribución GNU/Linux. Para tal fin, se incorporó al subsistema de procesamiento una *Single Board Computer* (o SBC por sus siglas en inglés) que funciona como unidad de procesamiento y media entre el subsistema de control y el de adquisición. Asimismo, esta unidad hace posible el procesamiento *in-situ* de los datos acústicos, que requiere operaciones computacionalmente demandantes tales como el cómputo de la transformada discreta de Fourier de segmentos de señal de longitud considerable.

El instrumental de adquisición se clasifica en diferentes categorías. Una de ellas es el instrumental específico, como es el caso de la placa de adquisición que se emplea para el registro de señales hidroacústicas. Otra categoría es la de instrumental integrado en los sensores, que incluye a los sensores de temperatura, el anemómetro y el GPS. En cualquier caso, los dispositivos de adquisición requieren ciertos protocolos de comunicación para acceder a los datos. A su vez, los datos deben ser almacenados en formato crudo, deben estar correlacionados entre sí (los datos acústicos deben estar asociados a fecha, hora, posición GPS, temperatura y velocidad del viento en el momento de la adquisición) y, en algunos casos, deben ser procesados (a manera de ejemplo, los datos acústicos deben ser convertidos a Niveles de Ruido Submarino).

Para llevar a cabo conjunto de actividades y tareas descritas es necesaria una adecuada integración de los subsistemas que componen a EAMMRA, de manera de obtener una comunicación sólida y eficiente de los mismos.

II. DESCRIPCIÓN DE LOS SUBSISTEMAS QUE COMPONEN EAMMRA

En esta sección se describen en detalle los componentes de cada subsistema que integra el prototipo EAMMRA. Asimismo, se explicitan las interfaces que cada módulo expone para la interacción con el resto del sistema.

II.1. Sensores

II.1.a. Hidrófonos y preamplificador

Los sensores utilizados en el prototipo de la estación de monitoreo EAMMRA para captar las señales acústicas son dos hidrófonos analógicos calibrados, fabricados por la empresa danesa Brüel & Kjær, modelo 8105. Se trata de transductores piezoeléctricos cerámicos que no poseen preamplificación interna. Estos hidrófonos poseen características destacadas que los hacen particularmente adecuados para el monitoreo de ruido submarino [Brüel & Kjær, 2017], a saber:

- Rango de frecuencia: 0.1 Hz a 160 KHz,
- Sensibilidad en recepción: -205 dB re 1 V/ μ Pa,
- Omnidireccional en todo el rango de frecuencia,
- Capacidad para operar a presiones hidroestáticas de 9.8 MPa, equivalente a 1000 m de profundidad,
- Completamente recubiertos por un copolímero de caucho sintético de acrilonitrilo y butadieno que evita la corrosión en el medio marino.

En la figura 2 se puede apreciar un dibujo del hidrófono con sus principales componentes y dimensiones mecánicas. Asimismo, en la figura 3 se presenta un patrón polar de directividad típico para el modelo de hidrófono seleccionado. Ambas figuras fueron tomadas de la hoja de datos del hidrófono.

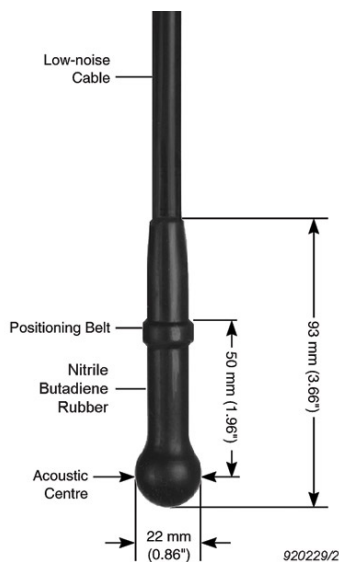


FIG. 2. Dimensiones mecánicas del hidrófono Brüel & Kjær modelo 8105.

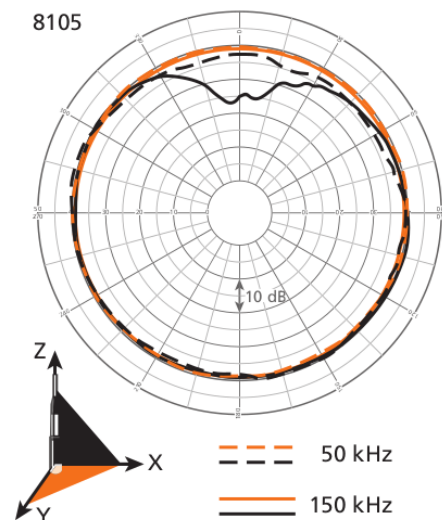


FIG. 3. Patrón de directividad típico del hidrófono Brüel & Kjær modelo 8105.

Los patrones reales de los hidrófonos que utiliza el prototipo, así como las curvas de respuesta en frecuencia vienen dadas en las respectivas hojas de calibración provistas por el fabricante. Cabe destacar

que en la información de calibración se ha tenido en cuenta el efecto no despreciable que introduce el cable que viene integrado al elemento sensor, y que consiste principalmente en agregar capacidad a la impedancia vista en el terminal de conexión. Los hidrófonos adquiridos para el desarrollo del prototipo poseen longitudes de cable de 50 m y 20 m, respectivamente.

El hidrófono utiliza un tipo de cable sumergible (*waterblocked*) de grado militar tipo DSS-2 especificado en el estándar MIL-DTL-915/8 [Military and Government Specs & Standards, 2010]. Se trata de un cable flexible, aislado y blindado, de dos hilos conductores para uso en electrónica, comunicaciones e instrumentación en equipamiento abordo de unidades navales. Los cables de ambos hidrófonos están terminados en un conector BNC no sumergible (*non-waterproof*).

Se diseñó y fabricó, como parte de las actividades desarrolladas dentro del Proyecto PIDDEF 32-14, un preamplificador para cargas capacitivas con un ancho de banda de aproximadamente 100 kHz, consistente con el ancho de banda de trabajo de los sensores [Cinquini *et al.*, 2018]. Las amplitudes involucradas tienen un máximo absoluto de 5 V, con valores que habitualmente no superarán el rango entre -1 y +1 V.

II.1.b. Sensores de Temperatura

Se utilizan para medir temperatura termómetros digital modelo DS18B20 del fabricante Maxim Integrated [Maxim Integrated, 2018]. En la figura 4 se puede apreciar un diagrama en bloques del sensor, en donde destaca la interfaz de tres cables con una línea bidireccional de datos (DQ) y dos cables para la alimentación (GND y V_{DD}).

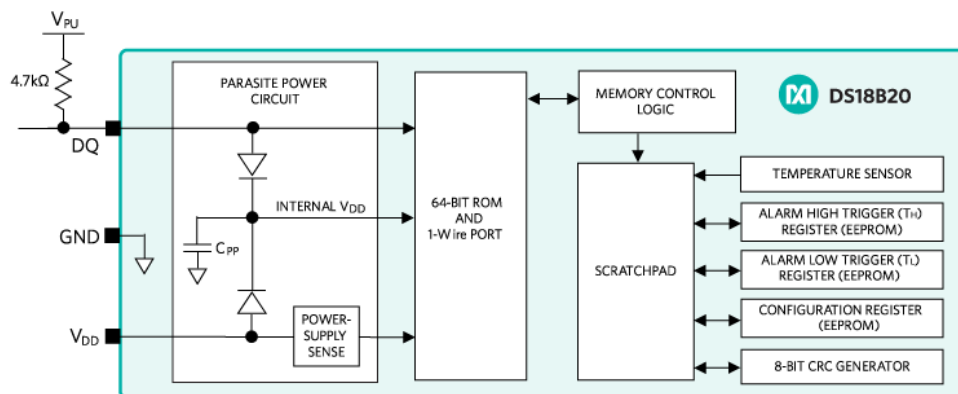


FIG. 4. Diagrama en bloques del sensor de temperatura¹.

El sensor admite una configuración de alimentación “parásita”, en la cual se energiza desde la línea de datos DQ a través de la carga de un capacitor interno, lo que habilita a prescindir de un hilo conductor en la interfaz. Como contrapartida, esta configuración requiere especial cuidado en la temporización de las comunicaciones sobre la línea de datos para asegurar que este capacitor interno, indicado como C_{pp} en la figura 4, no se descargue completamente, ya que de él depende la alimentación del dispositivo.

Asimismo, en este modo de alimentación es necesaria la utilización de un transistor a modo de “*strong pull-up*”, como indica la figura 5. Este transistor permite conectar el bus de datos directamente a V_{DD} durante las operaciones que más tiempo insumen como puede ser la conversión de un valor de temperatura, en la cual la línea de datos permanece en estado bajo de tensión y, consecuentemente, el capacitor no recibe carga.

El dispositivo opera sobre un bus maestro/multi-esclavo conocido como *1-wire*, un protocolo de comunicación propietario desarrollado por Dallas Semiconductor Corp.

¹Imagen tomada de <https://www.maximintegrated.com/en/images/qv/2812.png>

Este sensor se comercializa con distintas longitudes de cable, dentro de un encapsulado metálico sumergible, como se puede apreciar en la figura 6. Para el desarrollo del prototipo EAMMRA se adquirieron dos sensores sumergibles con 5 metros de cable y 2 metros de cable, respectivamente, pensados para registrar la temperatura a distintas profundidades de la columna de agua.

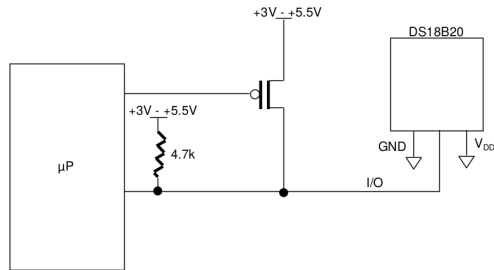


FIG. 5. Esquema de conexión en modo de alimentación parásita con *strong pull-up*.



FIG. 6. Sensor digital de temperatura DS18B20 con 5 metros de cable.

Cada sensor posee un código de identificación único de 64 bits que permite operar con múltiples sensores sobre un mismo bus tipo 1-wire. Mediante un algoritmo de descubrimiento se pueden identificar los sensores conectados al bus y controlarlos independientemente.

Otras características destacables son la posibilidad de configuración de resolución entre 9 y 12 bits, el uso de CRC (*cyclic redundancy check*) en las comunicaciones y la capacidad para definir umbrales de alarma para los valores de temperatura medidos, a partir de los cuales se podrían disparar eventos en el sistema.

II.1.c. Anemómetro y GPS

Se utiliza un anemómetro marca Gill Instruments, modelo WindSonic opt1 RS232. El modelo del GPS es BU-353S4, de la marca GlobalSat. Ambos establecen su comunicación a través de un protocolo serie, específicamente intercambiando mensajes NMEA, los cuales son accesibles mediante una terminal serie. En el caso del anemómetro, la interfaz física es RS232, mientras que el GPS se conecta a una interfaz USB.

II.2. Instrumental de adquisición

II.2.a. Placa de adquisición

Los requerimientos iniciales (propuestos en la formulación original del proyecto PIDDEF 32-14) establecían la necesidad de registrar hasta cuatro señales acústicas en forma simultánea, en un rango espectral desde 0 hasta 250 kHz inclusive, con una etapa de digitalización cuya figura de ruido fuese la más baja posible, dadas las muy bajas amplitudes provistas por los hidrófonos. Las exigencias de frecuencia de muestro, niveles de ruido propio, cantidad de canales y resolución en bits eran tales que su implementación como hardware *ad-hoc* (esto es, diseñado y fabricado en el DPA) excedía los alcances del proyecto PIDDEF 32-14 dentro del que está enmarcado el desarrollo del prototipo EAMMRA. Por tal motivo se decidió buscar en el mercado instrumental de adquisición comercial que cumpliera con los requerimientos necesarios.

Uno de las placas contempladas es la USB-6356, de National Instruments, entre cuyas características se citan:

- 8 canales diferenciales de muestreo simultáneo

- Frecuencia de muestro de 1 MHz por canal
- Ancho de banda de 500 kHz
- Resolución de 16 bits por canal
- Rango de tensiones de entrada: ± 1 V, ± 2 V y ± 10 V, con una incerteza de 0.31 mV (para ± 1 V a $f = 1$ kHz).
- Conexión a PC vía puerto USB 2.0.
- Librerías para C++, Python y Labview (entre otros).

En etapas intermedias del desarrollo del proyecto se acotó el ancho de banda a 100 kHz y la cantidad de señales simultáneas a 2 (lo que es equivalente a 2 hidrófonos). Esto, sumado a las restricciones de los *drivers* de la placa NI USB-6356 para ser utilizados en entornos con otros sistemas operativos que no fueran versiones de Windows, condujo a buscar un hardware alternativo, por un lado más económico, y por otro más versátil para acceder a los datos digitalizados. Se optó por la placa UMC204HD, de la marca Behringer, la cual cuenta con las siguientes características principales:

- 2 canales (no diferenciales) de muestreo simultáneo
- Frecuencia de muestreo máxima de 192 kHz por canal
- Ancho de banda 50 kHz (a -2 dB), ó 96 kHz (a -10 dB)
- Resolución de 24 bits por canal
- Rango de tensiones de entrada $\pm 0,5$ V (entrada en modo *inst*) y ± 15 V (entrada en modo *line*).
- Accesible mediante el driver genérico ALSA, provisto en distribuciones de sistemas operativos basados en Linux.

La placa UMC204HD carece de especificaciones de ruido propio, por tratarse de un hardware orientado a la grabación de audio en ámbitos semiprofesionales, y no orientado al ámbito de adquisición de señales para uso científico, por lo que fue necesaria una caracterización de los canales de entrada, la cual se incluye en (?).



FIG. 7. Placas de adquisición contempladas como parte del equipamiento que constituye el prototipo EAMM-RA. Izquierda: NI USB-6356 OEM. Derecha: Behringer UMC204HD.

II.2.b. Instrumental integrado en los sensores

En en casos del GPS, que no se trata de un sensor propiamente dicho, sino un dispositivo de recepción de datos (que ya están en formato digital, salvo por la capa física de transmisión inalámbrica), la recepción de la información y su conversión a una cadena de *bytes* que se envían por puerto serie se realiza dentro del dispositivo. El manual de usuario describe que los datos GPS son accesibles desde una PC convencional conectándose mediante una terminal serie (montada a través de una interfaz USB) a un *baudrate* fijo de 4800 baudios. La recepción de datos es permanente y automática, y bajo la forma de cadenas de mensajes que poseen ciertas palabras reservadas, la cual se denomina NMEA.

El anemómetro opera de una manera muy similar, ya que las señales analógicas que manipula son auxiliares, puesto que la estimación de la velocidad del viento se realiza mediante la medición del tiempo de vuelo de un pulso acústico entre un emisor y un receptor incorporados en el dispositivo y ubicados a una distancia prefijada. El tratamiento de los datos acústicos, y su conversión a módulo de velocidad y dirección del viento se realiza dentro del dispositivo, y los datos son accesibles por protocolo serie e intercambiando mensaje que responden al estándar NMEA.

II.3. Unidad de procesamiento

La utilización de instrumental de adquisición de tipo comercial condujo a la necesidad de interactuar con el mismo mediante dispositivos que funcionasen de manera similar a una PC, es decir, con sistema operativo de propósitos generales, debido a que dicho instrumental es accesible a través de *drivers* propietarios cerrados de los cuales no se dispone el código fuente. Se consideraron soluciones *one-board*, similares a la computadora Raspberry Pi. Finalmente se escogió el modelo Cubieboard 2 de la empresa Cubietech, por combinar un precio accesible, la posibilidad de adquirirla en el mercado local, y especificaciones técnicas que cubren las necesidades del proyecto para tareas de adquisición y procesamiento. Las mismas se citan a continuación:

- Procesador AllWinner A20, con arquitectura ARM Cortex-A7, *dualcore* a 1 GHz.
- Memoria RAM de 1 GB.
- NAND interna de 4 GB y slot para memoria micro SD de hasta 64 GB.
- Puertos de pines GPIO programables
- Puertos para comunicación, que utilizan protocolos spi, i2c, y uart.
- 2 Puertos USB 2.0
- Slot para HDD/SDD SATA.
- Salida de video HDMI
- Alimentación con 5V (corriente máxima 3 A).

A la vez, se valoró la existencia de una comunidad para soporte, e imágenes de adaptaciones de Debian y Ubuntu compatibles con el dispositivo. Adicionalmente, se verificó que existieran proyectos basados en programación en Python (sobretudo aquellos en los que se accede a los pines GPIO y los puertos serie).

II.4. Unidad de control

Para la unidad de control de EAMMRA se optó por la plataforma CIAA-NXP, una computadora *oneboard* de grado industrial desarrollada en el marco del Proyecto CIAA (Computadora Industrial Abierta Argentina), como un esfuerzo conjunto entre sectores académicos del sistema científico-tecnológico nacional y sectores industriales agrupados en la cámara empresaria CADIEEL [Proyecto CIAA, 2014]. Una vista superior de la plataforma puede apreciarse en la figura 8.

La plataforma CIAA-NXP es una computadora que reúne dos cualidades:

- Ser **Industrial**, ya que su diseño está preparado para las exigencias de confiabilidad, estrés térmico, vibraciones mecánicas, interferencia electromagnética, sobrecarga de tensión en las salidas, cortocircuitos, perturbaciones en la tensión de alimentación, etc., que demandan los productos y procesos industriales.
- Ser **Abierta**, ya que toda la información sobre su diseño de *hardware*, *firmware*, *software*, etc. está libremente disponible en internet bajo la licencia BSD, para que cualquiera la utilice según sus necesidades.

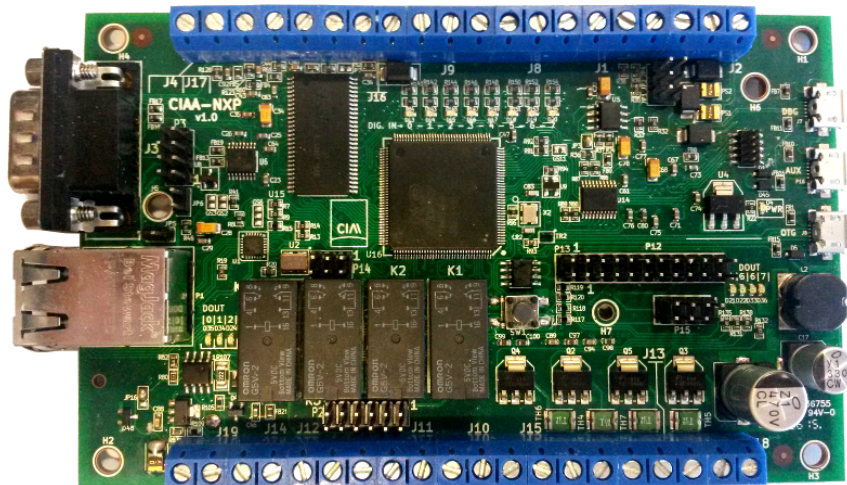


FIG. 8. Vista superior de la Computadora Industrial Abierta Argentina, CIAA-NXP.

Esta plataforma se compone de:

- CPU: Microcontrolador NXP LPC4337JBD144 (Dual-core Cortex-M4 + Cortex-M0 @ 204MHz).
- Debugger: USB-to-JTAG FT2232H. Soportado por OpenOCD.
- Memorias:
 - IS42S16400F - SDRAM. 64Mbit @ 143MHz.
 - S25FL032P0XMF1011 - Flash SPI. 32 Mbit, Quad I/O Fast read: 80 MHz.
 - 24AA1025 - EEPROM I2C. 1 Mbit, 400 kHz. Almacenamiento de propósito general, datos de calibración del usuario, etc.
 - 24AA025E48 - EEPROM I2C. 2 kbit, 400 kHz. Para implementación de MAC-Address o almacenamiento de propósito general.

- Entradas / salidas:
 - 8 entradas digitales opto-acopladas.
 - 4 entradas analógicas configurables por jumper 0-10V o 0-20mA
 - 4 salidas MOSFET Open-Drain de 24V, 1A.
 - 4 salidas a relé DPDT 24V, 2A.
 - 1 salida analógica configurable por jumper 0-10V o 0-20mA.
 - Conectores de expansión LV-GPIO, SPI, I2C.
- Interfaces de comunicación:
 - Ethernet con soporte PoE (requiere módulo de alimentación PoE)
 - USB On-The-Go
 - USB Device Auxiliar
 - RS232
 - RS485
 - CAN

II.5. Alimentación

La alimentación del prototipo EAMMRA es provista por una batería náutica de 12 V y 100 AH de capacidad, la cual se contempla que sea periódicamente cargada por paneles solares.

III. DESCRIPCIÓN DE LAS ACTIVIDADES DEL PROTOTIPO

III.1. Ciclo de funcionamiento sintético

III.1.a. Ciclo completo

III.1.b. Ciclo simplificado

III.1.c. Presencia de usuario conectado por SSH

III.1.d. Casos de falla

III.2. Diagrama de flujo completo

IV. MÓDULOS PRINCIPALES DE SOFTWARE

En esta sección se describe el software en su conjunto (sistema operativo, aplicaciones, módulos, *scripts*, etc) que se encuentra operativo en cada uno de los subsistemas del prototipo EAMMRA.

IV.1. Unidad de procesamiento

La unidad de procesamiento es una computadora *single-board* capaz de correr un sistema operativo de tipo GNU Linux, por la cual el sistema operativo en sí mismo constituye uno de las partes a considerar. Sobre el mismo corren ciertos módulos, escritos en Python, que interactúan con *drivers* para realizar las actividades de adquisición. Otros módulos se encargan del procesamiento de los datos, y otros de la comunicación con la Unidad de Control.

IV.1.a. Sistema operativo y lenguaje de programación utilizados

Se escogió un O.S. de tipo GNU Linux, en particular una distribución basada en Debian con un Kernel adaptado para procesadores tipo ARM. Específicamente, se instaló ARMBIAN Stretch con Kernel Mainline 4.19, con soporte para procesadores AllWinner A20 sin entorno gráfico. La instalación de la imagen se realizó en la tarjeta SD, pues esta versión no tiene soporte para la NAND interna. Se verificó que estuviera instalado el *driver* genérico ALSA mediante el cual es posible interactuar con periféricos de la clase *USB-Audio*. Se instaló Python 3 junto con los siguientes módulos específicos:

- **numpy**: permite procesamiento de arrays de números reales o complejos, en forma similar al se realiza usando MATLAB. Incluye paquetes para implementar diversas variantes de Transformada Rápida de Fourier (FFT).
- **wavio**: permite la manipulación (escritura y lectura) de archivos en formato WAV con *bitrate* mayores que 16 bits.
- **pyserial**: permite enviar y recibir comandos RS232 a través de puertos serie.
- **pynmea**: *parser* de mensajes según el estándar NMEA para interpretación de datos obtenidos de GPS.
- **pyaudio**: librería que permite leer o escribir fragmentos o *chunks* de datos correspondientes a señales temporales a través de las entradas o salidas de línea de placas de audio, entre ellas las pertenecientes a la clase *USB-Audio*.

IV.1.b. *Scripts* de Python

Todos los *scripts* dedicados a las funciones específicas de la Unidad de Procesamiento se encuentran en la ruta `/home/cubie` presente en la Unidad.

- *Script* de adquisición (`recorder.py`): procede a adquirir (grabar) señales de audio provenientes de los hidrófonos que están coenctados a los 2 canales de la placa UMC204HD. La duración total de cada par de señales adquiridas es de 1 minuto, con una frecuencia de muestro de 192 kS/s y una resolución de 24 bits. Para realizar esta tarea, el *script* interactúa principalmente con la placa de audio UMC204HD utilizando el módulo *pyaudio*, el cual a su vez utiliza el driver genérico ALSA. De manera secundaria interactúa con el GPs, empleando los módulos *pynmea* y *pyserial*, para obtener la fecha, hora, latitud y longitud al momento de la adquisición. Almacena la adquisición

archivos con formato `.wav` estéreo, titulados con los datos GPS mencionados, y ubicados en la ruta `/home/cubie/datos`.

- *Script* de procesamiento (`pot_audios_2019_test.py`): recibe como *input* un archivo de audio estéreo (generado luego de la ejecución del *script recorder.py*). Genera un estimador de la Densidad Espectral de Potencia (PSD) de cada uno de los canales contenidos en el archivo de audio y luego se estima la potencia acústica en bandas de frecuencia, para luego almacenar la información resultante en un archivo de texto etiquetado, en el título del archivo, con la fecha, hora, latitud y longitud al momento de la adquisición. El archivo se organiza en 2 columnas de 50?? elementos cada una. Cada elemento simboliza la potencia acústica, en dB referido a $1 \mu\text{Pa}^2$ contenido en una determinada banda de frecuencia, entre 0 Hz y 96 kHz. Las mismas están especificadas en el archivo `bandas_tercio_octava.txt`, ubicado en `/home/cubie`. El *script* tiene en cuenta la respuesta en frecuencia del preamplificador, de la placa de audio y la sensibilidad en recepción de cada uno de los hidrófonos.
- *Script* de comunicación (`cinquini_serial.py`): encuesta permanentemente al puerto serie para establecer comunicación con la Unidad de Control. Responde a comandos de *handshake*, pedido de datos GPS, adquisición, procesamiento, solicitud de datos procesados y pedido de suspensión (modo de bajo consumo). Tiene capacidad de manipulación de ciertos casos de fallas, tales como:
 - fallas en la adquisición: si la placa de adquisición no se encontrara en buenas condiciones, o se hubiese interrumpido la alimentación eléctrica, o hubiese un enlace de comunicación defectuoso entre la placa y la Unida de procesamiento, ocurriendo estos eventos antes de una solicitud de adquisición, el *script recorder.py* no escribe el archivo, pero el ciclo de funcionamiento no se interrumpe. De manera similar, si la interrupción en la comunicación con la placa (por el motivo que fuere) se produce durante la adquisición, el archivo de audio se escribe hasta el último instante en que hubo un flujo de adquisición exitoso. A la vez, si se recibe de la Unidad de Control solicitando una adquisición mientras existe una en curso, la solicitud se ignora.
 - fallas en el procesamiento: si, al recibirse una solicitud de procesamiento, el *script* `pot_audios_2019_te` no encuentra archivos de audio con fecha y hora dentro de la última hora, el archivo de texto no se escribe, y el ciclo no se interrumpe. Luego, cuando se recibe una solicitud de datos procesados, la Unidad de Procesamiento envía un comando de ausencia de datos a la Unidad de Control, para que, en caso de prolongarse esta situación durante un tiempo mayor que cierto umbral, se genere una señal de alarma. Asimismo, si se recibe una solicitud de procesamiento mientras existe una en curso, la solicitud se ignora.

IV.1.c. Interfaz de usuario

IV.2. Unidad de control

Para el firmware de la unidad de control se optó por una arquitectura de tipo reactiva o *event-triggered* como se conoce en la literatura especializada, en la cual el sistema permanece la mayor parte del tiempo esperando la llegada de un evento para procesar. Los eventos que puede recibir el sistema pueden ser tanto de naturaleza sincrónica como asincrónica. Este tipo de arquitecturas resulta conveniente para su aplicación en EAMMRA debido a que se puede hacer uso del tiempo en que el sistema no tiene eventos para procesar para poner la unidad de control en un modo de bajo consumo eléctrico y de esta manera extender la duración de las baterías.

Se implementó un sistema operativo de tiempo real (RTOS, por sus siglas en inglés) embebido sencillo haciendo uso de un esquema cooperativo para la política de planificación de tareas. La programación se hizo enteramente en lenguaje C bajo el estándar ISO/IEC 9899:1990, también referido como C90.

Se siguió el patrón de diseño descrito en el Informe Técnico AS 06/18 [Bos, 2018] que consiste fundamentalmente en encapsular la funcionalidad asociada a cada sensor y/o periférico en un módulo de *software*. La lógica principal de cada módulo se controla mediante una máquina de estados finitos o MEF, un modelo computacional basado en la teoría de autómatas [Samek, 2009].

Se adicionaron dos nuevos módulos a los existentes para controlar el reloj de tiempo real y la interacción con la unidad de procesamiento, nombrados RTC y CUBIE, respectivamente.

IV.2.a. Módulo RTC

La estructura de control del módulo RTC se encuentra definida con los siguientes valores de inicialización:

```
static module_t module_rtc = {
    .coreID      = CPUID_M4,
    .moduleID     = RTC,
    .moduleName   = "[RTC]",
    .eventHandler = rtc_handler,
    .period       = TIMER_DISABLED,
    .status       = DISABLED,
    .runme        = FALSE
};
```

Toda la interacción con el módulo se realiza a través de la función manejadora de eventos `rtc_handler`, que implementa la actualización de la máquina de estados finitos del módulo y se define con prototipo:

```
void rtc_handler(const ipcex_msg_t * msg );
```

esta función es la encargada de procesar las distintas señales que recibe el módulo e interpretar los parámetros que puedan estar opcionalmente adjuntos en las señales. El argumento de tipo `ipcex_msg` se define como:

```
typedef struct __ipcex_msg {
    struct {
        cpuID_t cpu;
        moduleID_t pid;
    } id;
    signal_t signal;
    void * param;
} ipcex_msg_t;
```

En la figura 9 se puede apreciar el diagrama de estados del módulo junto con las señales que producen las distintas transiciones. También se puede observar el punto de entrada a la MEF, indicado con un círculo negro.

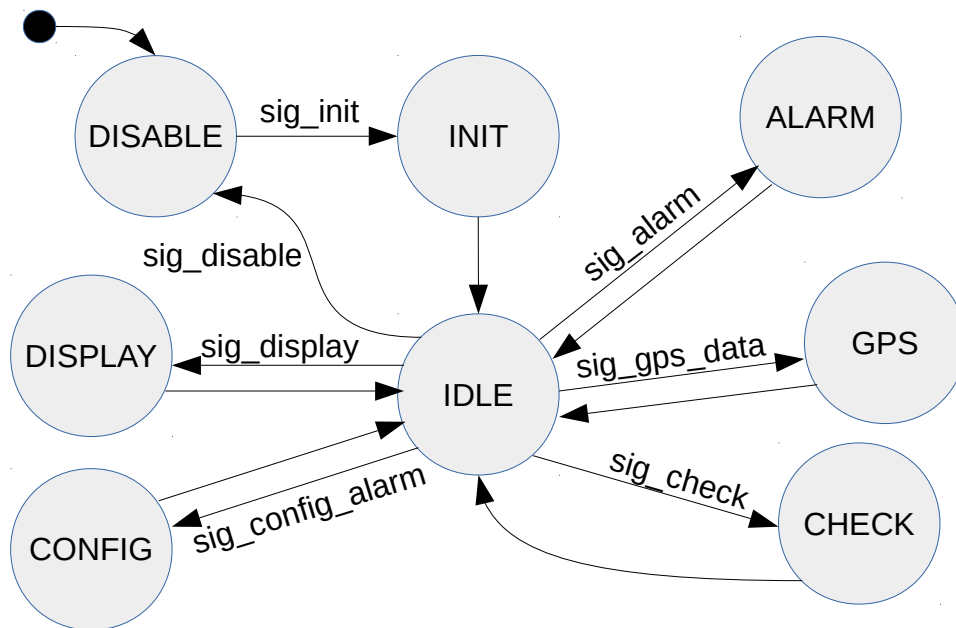


FIG. 9. Diagrama de estados de la MEF del módulo RTC.

TABLA 1. Descripción de los estados de la MEF principal del módulo RTC. Se indican las señales que puede recibir junto con las acciones y actividades más relevantes de cada estado.

Estado	Señal para cambio	Acciones y actividades
DISABLE	Reset o sig_disable	Desinicializar el RTC. Deshabilitar la interrupción RTC. Deshabilitar el TIMER interno. Cambiar el estado del módulo de READY a DISABLE.
INIT	sig_init	Inicializar el RTC. Habilitar la interrupción RTC. Configurar frecuencia de interrupción en 1 Hz. Cambiar el estado del módulo de DISABLE a READY.
IDLE	Transición incondicional	Cambiar estado del módulo de PROCESSING a READY.
CONFIG	sig_config_alarm	Activa una alarm con un <i>offset</i> en minutos de la hora actual.
CHECK	sig_check	Realizar autochequeo e informar al módulo de control
GPS	sig_gps_data	Actualizar la fecha y la hora del RTC con el GPS
ALARM	sig_alarm	Enviar una señal sig_timeout al módulo CUBIE.
DISPLAY	sig_display	Actualizar la hora en la interfaz de usuario.

Todas las transiciones terminan incondicionalmente, luego de ejecutar las sentencias y llamadas a funciones específicas de cada estado, en el estado IDLE, donde el módulo queda disponible para recibir nuevas señales. En la tabla 1 se listan los estados del módulo junto con las señales necesarias alcanzarlos y una breve descripción de las principales acciones y actividades que en ellos se ejecutan.

IV.2.b. Módulo CUBIE

El módulo de *software* CUBIE se utiliza como controlador de la comunicación entre la unidad de control CIAA y la unidad de procesamiento CUBIE. La comunicación se implementa mediante un protocolo serie a través del periférico UART (siglás en inglés de *Universal Asynchronous Receiver-Transmitter*). La capa física de la comunicación se implementa mediante un cable USB, para lo cual se hace uso de un *chip* FT2232 de la empresa FTDI, integrado en la platadorma CIAA, que adapta los niveles de tensión eléctrica de UART a USB [Future Technology Devices International Ltd., 2016].

La estructura de control del módulo CUBIE se encuentra definida con los siguientes valores de inicialización:

```
static module_t module_CUBIE = {
    .coreID      = CPUID_M4,
    .moduleID    = CUBIE,
    .moduleName  = "[CUBIE]",
    .eventHandler = cubie_handler,
    .period      = TIMER_DISABLED,
    .status      = DISABLED,
    .runme       = FALSE
};
```

Toda la interacción con el módulo se realiza a través de la función manejadora de eventos `rtc_cubie`, definida con el siguiente prototipo, `void rtc_handler(const ipcex_msg_t * msg)`. Esta función implementa la actualización de la máquina de estados finitos del módulo y es la encargada de procesar las distintas señales que recibe el módulo e interpretar los parámetros que puedan estar adjuntos opcionalmente en las señales.

En la figura 10 se muestra el diagrama de estado de la máquina de estados finitos principal del módulo donde se pueden apreciar las diferentes señales que puede recibir el módulo y las transiciones que éstas generan. Se observa el punto de entrada a la MEF indicado con un círculo negro y que ésta se inicia en el estado `DISABLE`.

La lógica de funcionamiento es idéntica a la de los otros módulos que componen el *firmware* de la CIAA, la MEF se inicial deshabilitada y al recibir una señal de inicialización `sig_init` pasa al estado `INIT` en el cual se ejecutan las primeras rutinas de configuración del módulo y luego se pasa incondicionalmente al estado `IDLE`. En el estado `IDLE`, que es donde la MEF pasa la mayor parte del tiempo, se espera la recepción de una señal. Cuando se recibe una señal se pasa al estado correspondiente y cuando el procesamiento específico finaliza se produce una transición incondicional al estado `IDLE` a la espera de nuevas señales.

El módulo CUBIE comparte recursos y utiliza un mismo medio físico que el módulo HMI, la UART USB, con lo cual no pueden estar los dos habilitados simultáneamente. Si bien ambos módulos se inician deshabilitados, el módulo CUBIE se encuentra en una lista de módulos a activar por el módulo de control cuando el sistema se energiza. En módulo HMI en cambio, debe esperar que el módulo CUBIE sea deshabilitado por la presencia de un operador humano conectado a la placa de adquisición, para que le sea enviada una señal `sig_init` que lo inicialice. Para volver a inicializar el módulo CUBIE sin reiniciar el sistema, el operador humano debe deshabilitar el módulo HMI desde la interfaz de usuario para que

automáticamente se envíe una señal *sig_init* al módulo CUBIE.

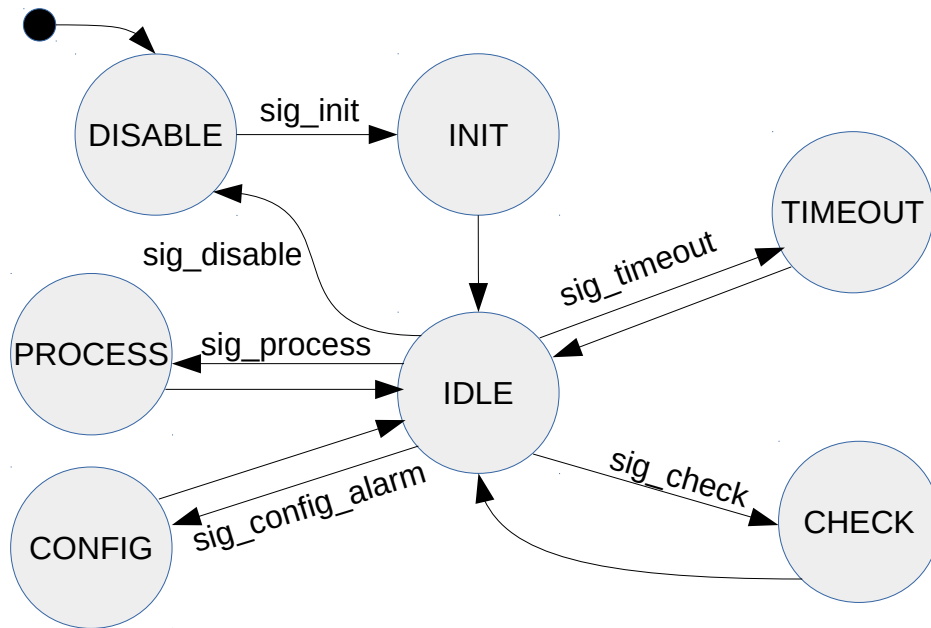


FIG. 10. Diagrama de estados de la MEF del módulo CUBIE.

En la tabla 2 se incluyen los diferentes estados que puede tomar la MEF principal del módulo, las señales que originan las transiciones y una breve descripción de las acciones que suceden en cada uno de estos estados.

TABLA 2. Descripción de los estados de la MEF principal del módulo CUBIE. Se indican las señales que puede recibir junto con las acciones y actividades más relevantes de cada estado.

Estado	Señal para cambio	Acciones y actividades
DISABLE	Reset o sig_disable	Desinicializar la UART. Deshabilitar la interrupción UART. Enviar señal de inicialización al módulo HMI. Cambiar el estado del módulo de READY a DISABLE.
INIT	sig_init	Inicializar la UART. Habilitar la interrupción UART. Configurar un GPIO para sacar al CUBIE de estado bajo consumo. Cambiar el estado del módulo de DISABLE a READY.
IDLE	Transición incondicional	Cambiar estado del módulo de PROCESSING a READY.
CONFIG	sig_config_alarm	Activa una alarm con un <i>offset</i> en minutos de la hora actual.
CHECK	sig_check	Realizar autochequeo e informar al módulo de control
PROCESS	sig_process	Actualizar la MEF de comunicación con la placa de adquisición.
TIMEOUT	sig_timeout	Enviar una señal sig_process al módulo CUBIE.

IV.3. Protocolos específicos

IV.3.a. Temperatura - *1 wire*

El protocolo *1-Wire* es una marca registrada de Maxim/Dallas Semiconductor y permite la interacción con diferentes dispositivos de este fabricante, incluyendo baterías, dispositivos de gestión térmica, memoria, entre otros. Es un protocolo del tipo *master/slave* con la posibilidad de tener múltiples dispositivos esclavos en un mismo *bus* arbitrado por un único maestro.

El protocolo se llama *1-Wire* porque usa un sólo cable conductor para transmitir datos. La arquitectura de un cable utiliza una resistencia de *pull-up* para extraer energía eléctrica de la línea de datos en el lado del maestro. Utiliza lógica CMOS/TTL y funciona a una tensión de alimentación que puede variar de 2.8 V a 6 V.

Cada dispositivo, maestro o esclavo se conecta a la línea de datos a través de una conexión tipo *open-drain* o *tri-state* lo que permite “liberar” el bus cuando el dispositivo no se encuentre transmitiendo para que pueda ser usado por otro dispositivo.

El maestro inicia y controla todas las operaciones *1-Wire*. Tanto el dispositivo que actúa como maestro como el que actúa como esclavo pueden ser receptores y transmisores, pero sólo transfieren en una dirección a la vez, en configuración semidúplex. Las transacciones se realizan en ranuras de tiempo y están orientadas a lectura y escritura de bits con el envío del bit menos significativo primero.

No es necesario compartir una señal de *clock* única entre maestro y esclavo ya que cada parte utiliza su propio reloj y la comunicación está sincronizada por el flanco descendente del maestro.

Las transacciones para acceder al termómetro digital DS18B20 a través del bus *1-wire* siguen el siguiente orden lógico:

1. Inicialización.
2. ROM *command* seguido del intercambio de información correspondiente.
3. Ejecución de función por parte del DS18B20, seguido del intercambio de información correspondiente.

Es importante que todas las transacciones sigan la secuencia lógica enumerada ya que el dispositivo no responderá si algún paso es omitido o éstos se ejecutan en otro orden.

Todas las transacciones comienzan con una secuencia de inicialización que consiste en un pulso de *reset* emitido por el dispositivo maestro controlador del bus *1-wire*, seguido por un pulso de presencia emitido por el o los esclavos. El pulso de presencia le permite al controlador saber que los dispositivos esclavos están conectados al bus y listo para operar. Más información sobre la temporización de las señales puede encontrarse en la hoja de datos del dispositivo DS18B20 [Maxim Integrated, 2018].

Luego que el maestro detecta un pulso de presencia puede emitir un comando ROM. Estos comandos operan sobre la base de códigos de 64 bits únicos para cada tipo de dispositivo y le permiten al bus *master* identificar individualmente a un dispositivo. También le permiten al controlador del bus obtener información sobre cuántos y de qué tipo son los dispositivos conectados si más de uno estuviera presente en el bus.

El controlador del bus debe emitir un comando ROM válido antes de poder enviar un comando para ejecutar una función del DS18B20.

IV.3.b. GPS. Mensajes NMEA sobre protocolo serie

IV.3.c. Anemómetro

V. CONEXIONADO FÍSICO

V.1. Diagrama funcional en bloques

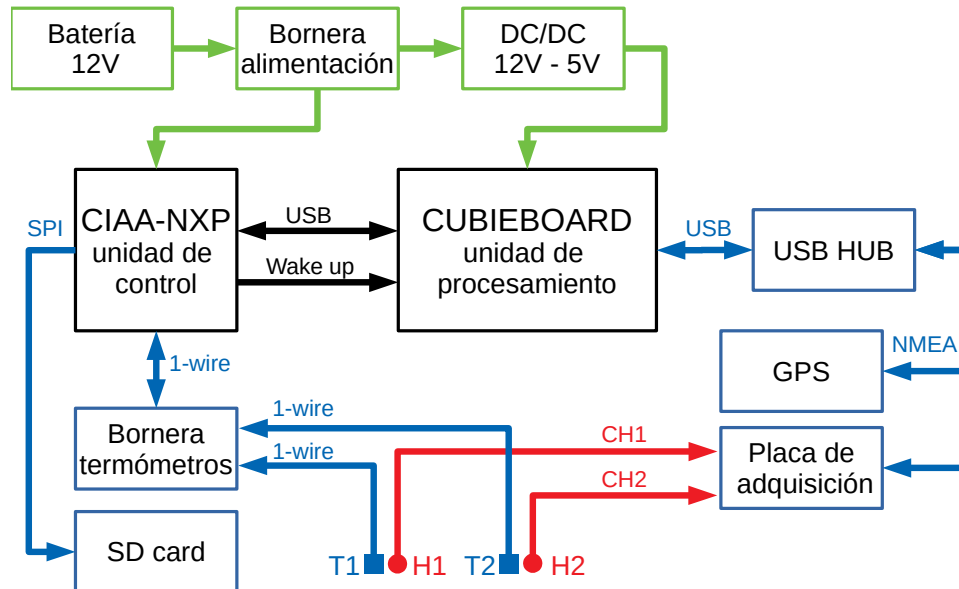


FIG. 11. Diagrama funcional en bloques.

V.2. Puertos y conectores

V.2.a. Sensores

V.2.b. Adquisición

V.2.c. Unidad de procesamiento

V.2.d. Unidad de control

V.2.e. Alimentación

V.3. Housing y diseño mecánico

VI. PRUEBAS DE LABORATORIO

VI.1. Pruebas de uso y funcionamiento

VI.2. Estudio de consumo y duración de la batería

VI.3. Estudio de llenado de la memoria SD

VI.4. Casos de falla

VI.5. Diseño de solución óptima en función de ciclos de batería y disco

VII. CONCLUSIONES

REFERENCIAS

- Bos, P. (2018). “*Sistema de control para estación autónoma marítima de monitoreo de ruido ambiente*”. INF. TEC. AS 06/18, DIIV, UNIDEF.
- Brüel & Kjær (2017). *Product Data: Hydrophon type 8103, 8104, 8105 and 8106*. Brüel & Kjær, Sound & Vibration Measurement A/S.
- Cinquini, M., Bos, P., Prario, I., y Blanc, S. (2018). “*Diseño y fabricación de un preamplificador para hidrófonos*”. INF. TEC. AS 05/18, DIIV, UNIDEF.
- Future Technology Devices International Ltd. (2016). *FT2232H Dual High Speed USB to Multipurpose UART/FIFO IC*. Version 2.5.
- Maxim Integrated (2018). *DS18B20 Programmable Resolution 1-wire Digital Thermometer*. Rev. 5.
- Military and Government Specs & Standards (2010). *CABLE, ELECTRICAL, FOR SHIPBOARD USE, GENERAL SPECIFICATION FOR*. Naval Publications and Form Center. MIL-DTL-915/8.
- Proyecto CIAA (2014). Computadora industrial abierta argentina. Disponible: 2019-04-29.
URL <http://proyecto-ciaa.com.ar/devwiki/doku.php?id=start>
- Samek, M. (2009). *Practical UML Statecharts in C/C++: Event-Driven Programming for Embedded Systems*. Electronics & Electrical. Taylor & Francis.
URL <https://books.google.com.ar/books?id=XrWPslzH9WoC>

APÉNDICES

I. *Scripts* computacionales.

II. Circuitos de electrónica adicional (borneras).

III. Especificaciones de partes constitutivas.

IV. Planos.